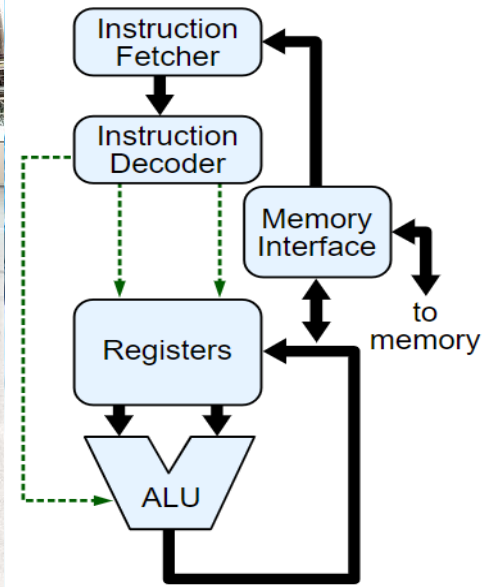
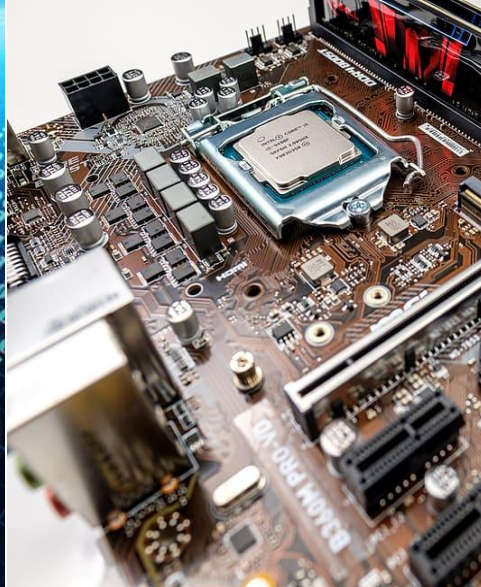




Overview of this Lecture

- ▶ This lecture will introduce some further MIPS instructions and show by example how these can be combined to carry out different tasks.
- ▶ We will test several of these on MARS.

Bit Manipulation Example

Say we have the binary for *an instruction* stored in \$4:

\$4 = 1010110111101000100000000000000000

By the way, instruction is "sw \$8, 0x8000 (\$15)"

TASK: Write assembly to determine the value of the register rs for this instruction.

We have a binary representation of the instruction:

\$4 = 1010110111101000100000000000000000

Useful intermediate:

0000000111100000000000000000000000 **HOW ?**

We want:

\$6 = 000000000000000000000000000000000111

Bit Manipulation Example

We have the binary representation:

\$4 = 10101101111010001000000000000000000

What
Operation

?

We can use a *bit mask*:

\$5 = 00000011111000000000000000000000000

```
lui $5, 0x03e0
```

\$6 = 000000001111000000000000000000000000

```
and $6, $4, $5
```

Shift Right
by 21 bits.

What
Operation

?

```
srl $6, $6, 21
```

We want:

\$6 = 00000000000000000000000000000000000001111

Can you think of a different way of doing it ?

Comparing registers with fixed numbers (for example, is \$4 < 10 ?)

We can use a “set comparison” instruction:

`slti rt, rs, value` # Set less than (signed)

If (rs < value) then set rt = 1 else set rt = 0

`sltiu rt, rs, value` # Set less than (unsigned)

If (rs < value) then set rt = 1 else set rt = 0

THEN we use *beq* or *bne* against \$0 to test result register rt after *set*.

Multiple ways to compute an average

```
# Calculating average (Version 1)
```

```
li $8, 173
```

```
li $9, 191
```

```
addu $10, $8, $9
```

```
srl $10, $10, 1
```

```
li $v0, 36 #syscall 36 -- print unsigned int
```

```
move $a0, $10 # a0 = $10
```

```
syscall #print $10
```

Multiple ways to compute an average

```
# Calculating average (Version 1)
```

```
li $8, 3501006752
```

```
li $9, 794337423
```

```
addu $10, $8, $9
```

```
srl $10, $10, 1
```

```
li $v0, 36 #syscall 36 -- print unsigned int
```

```
move $a0, $10 # a0 = $10
```

```
syscall #print $10
```

Multiple ways to compute an average

```
# Calculating average (Version 2)
```

```
li $8, 3501006752
```

```
li $9, 794337423
```

```
and $10, $8, $9
```

```
xor $11, $8, $9
```

```
srl $11, $11, 1
```

```
addu $10, $10, $11
```

```
li $v0, 36 #syscall 36 -- print unsigned int
```

```
move $a0, $10 # a0 = $10
```

```
syscall #print $10
```


What is going on?

- ▶ We want to compute the average of two registers

Cannot add
these up!

```
$8 = 11010000101011010001111110100000
$9 = 00101111010110001010000010001111
$10 = 00000000000010000000000010000000
```

and \$10, \$8, \$9

```
$8 = 11010000101011010001111110100000
$9 = 00101111010110001010000010001111
$11 = 11111111111101011011111100101111
```

xor \$11, \$8, \$9

Mathematical bitwise
identity:

$$x + y = 2 * (x \& y) + x \wedge y$$

Therefore:

$$\frac{x + y}{2} = (x \& y) + \frac{x \wedge y}{2}$$

srl \$11, \$11, 1
add \$10, \$10, \$11

Lecture Summary

- ▶ We discussed *I*-mmediate instructions
- ▶ Hopefully, we have demonstrated how different MIPS instructions can be combined to accomplish different tasks.
- ▶ The MIPS reference card(s) provide an *overview* of the most important MIPS instructions.
- ▶ Appendix A of Patterson & Hennessey's book provides a *comprehensive reference to all MIPS instructions* (available as a free PDF from the link on the Moodle page)